

RÉPUBLIQUE DU BÉNIN



MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEUR ET
DE LA RECHERCHE SCIENTIFIQUE



UNIVERSITÉ D'ABOMEY-CALAVI (UAC)

*INSTITUT DE FORMATION ET DE RECHERCHE EN
INFORMATIQUE (IFRI)*

MASTER 1 : Big Data

Travaux pratiques n° 4

Clustering IoT par K-Means MapReduce

Modélisation parallèle sur capteurs distribués — 5 serveurs régionaux

Génie Logiciel

Réalisé par :

DOSSOU Persévérance
LOKOSSOU Iréné
AMOUSSOU Gisé
MVOGO Bandolo Samira Anaïs
GOUDALO Brice

Sous la supervision de :

Dr John AOGA

Année académique : 2025-2026

Table des matières

1	Introduction	2
2	Modélisation du problème	2
2.1	Représentation des données	2
2.2	Pourquoi le parallélisme est-il applicable ?	2
2.3	Architecture générale du système	3
3	Le pipeline MapReduce appliqué au K-Means	3
3.1	Phase MAP + COMBINE : calcul local sur chaque serveur	3
3.2	Phase SHUFFLE et REDUCE	3
3.3	Efficacité du Combiner	4
3.4	Initialisation K-Means++	4
4	Implémentation et résultats préliminaires	4
4.1	Organisation des modules	4
4.2	Code du Mapper-Combiner et du Reducer	5
4.3	Métriques de qualité et critères d'arrêt	6
4.4	Exemple concret : 6 capteurs sur 2 serveurs	6
5	Résultats et performances	6
5.1	Benchmark sur données réelles	6
5.2	Scalabilité — montée en charge	7
5.3	Discussion des résultats	7
5.4	Qualité des clusters produits	8
5.5	Limites et perspectives	8
6	Conclusion	8

1 Introduction

Les réseaux de capteurs IoT (*Internet of Things*) génèrent des volumes de données croissants, répartis sur plusieurs sites géographiques. Dans ce contexte, regrouper automatiquement des milliers de capteurs en zones aux comportements similaires constitue un problème central de l'analyse Big Data.

L'algorithme de clustering K-Means est une solution classique à ce problème. Cependant, son exécution séquentielle sur un seul processeur atteint rapidement ses limites lorsque le volume de données augmente. Le paradigme **MapReduce** offre une alternative : en distribuant les calculs sur plusieurs nœuds, il permet de traiter de grands volumes de données tout en conservant la précision de l'algorithme d'origine.

Ce rapport présente l'implémentation et l'évaluation du K-Means MapReduce appliqué à un réseau IoT simulé de cinq serveurs régionaux (Nord, Centre, Sud, Ouest, Est), chacun stockant les mesures de 300 capteurs, soit 1 500 capteurs au total. Les expériences ont été conduites sur une machine à 4 CPU logiques. Le rapport s'organise comme suit : la section 2 modélise le problème et décrit l'architecture parallèle ; la section 3 détaille le pipeline MapReduce ; la section 4 présente l'implémentation ; la section 5 analyse les performances mesurées ; la section 6 conclut.

2 Modélisation du problème

2.1 Représentation des données

Chaque capteur est décrit par un vecteur de quatre mesures physiques collectées en continu. Ces quatre dimensions constituent l'**espace de features** dans lequel l'algorithme K-Means calcule les distances et identifie les clusters.

Feature	Unité	Plage	Signification
temperature	°C	13–38	Température ambiante
humidity	%	30–90	Humidité relative
vibration	m/s ²	0.3–3.0	Vibrations mécaniques
network_traffic	kbps	40–700	Trafic réseau local

2.2 Pourquoi le parallélisme est-il applicable ?

À chaque itération du K-Means, l'opération la plus coûteuse est l'**affectation** de chaque capteur au cluster dont le centroïde est le plus proche. Cette affectation est calculée par la distance euclidienne :

$$\text{cluster}(p) = \arg \min_{k=1}^K \|p - c_k\|_2$$

Le point essentiel est que ce calcul est *totalelement indépendant d'un capteur à l'autre* : le résultat pour le capteur i ne dépend pas du capteur j . Il est donc possible de répartir les capteurs sur plusieurs workers qui effectuent ces calculs simultanément, sans coordination

ni échange de données entre eux. Cette propriété est le fondement de la parallélisation par MapReduce.

2.3 Architecture générale du système

L'architecture retenue place les données au plus près de leur source : chaque serveur régional conserve ses 300 capteurs localement et exécute sa phase de calcul sans transfert préalable des données brutes. Seuls les résultats intermédiaires agrégés transitent vers le nœud central.

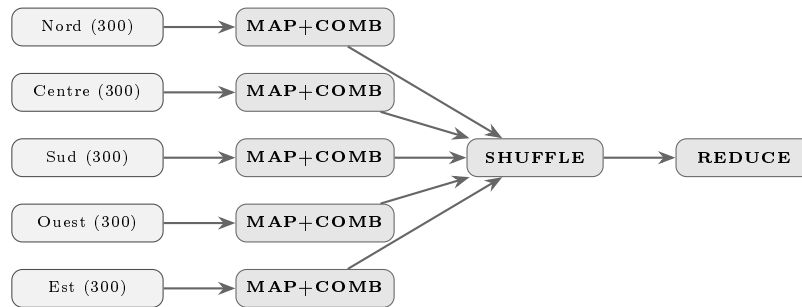


Figure 1: Pipeline MapReduce : 5 serveurs en parallèle → Shuffle → Reducer

3 Le pipeline MapReduce appliqué au K-Means

Une itération complète du K-Means MapReduce se déroule en trois phases enchaînées : MAP+COMBINE (exécuté en parallèle sur chaque serveur), SHUFFLE (regroupement des résultats) et REDUCE (calcul des nouveaux centroïdes). Ces phases se répètent jusqu'à convergence.

3.1 Phase MAP + COMBINE : calcul local sur chaque serveur

La phase MAP est exécutée par un **mapper** sur chaque serveur régional. Pour chaque capteur de son lot de données, le mapper calcule la distance euclidienne aux K centroïdes courants et assigne le capteur au cluster le plus proche. Ce travail est effectué sur les données locales, sans communication inter-serveurs.

Le **Combiner** agrège ensuite localement les capteurs du même cluster : au lieu de 300 enregistrements, chaque serveur envoie seulement K sommes partielles (somme des features + compteur). Ceci réduit le trafic réseau d'un facteur $300/K = 100\times$ avec $K = 3$.

3.2 Phase SHUFFLE et REDUCE

Le Shuffle regroupe les partiels par cluster. Le Reducer calcule ensuite le nouveau centroïde de chaque cluster en combinant les sommes partielles :

$$c_k^{\text{new}} = \frac{\sum_w (\text{somme}_w^{(k)})}{\sum_w n_w^{(k)}}$$

Cette opération est **mathématiquement exacte** et produit le même résultat que l'algorithme séquentiel.

3.3 Efficacité du Combiner

Plutôt que de transmettre $N \times W$ enregistrements bruts (ex. $300 \times 5 = 1\,500$), le Combiner agrège localement et n'envoie que $K \times W$ sommes partielles ($3 \times 5 = 15$). Facteur de réduction : $N/K = 100\times$ à l'échelle de base. À 50 000 capteurs par serveur (10k/serveur), ce gain monte à $\times 3\,333$, d'où l'importance du Combiner pour l'efficacité réseau à grande échelle.

3.4 Initialisation K-Means++

La qualité du clustering dépend du choix des centroïdes initiaux. L'implémentation utilise l'heuristique **K-Means++** : le premier centroïde est choisi aléatoirement, puis chaque centroïde suivant est sélectionné avec une probabilité proportionnelle au carré de sa distance au centroïde existant le plus proche. Cette stratégie réduit le risque de convergence vers des minima locaux et améliore la qualité finale des clusters.

4 Implémentation et résultats préliminaires

4.1 Organisation des modules

Le pipeline est implémenté en Python avec une séparation claire des responsabilités.

Module	Responsabilités
generate_data.py	Génération des données IoT simulées : 5 serveurs régionaux, 300 capteurs/serveur, 3 profils de comportement + dataset stress 50k
utils.py	Fonctions utilitaires partagées : distance euclidienne, normalisation z-score, SSE, K-Means++, chargement CSV
kmeans_mr_job.py	Cœur MapReduce : mapper-combiner, shuffle, reducer, orchestrateur d'itération, <code>multiprocessing.Pool</code>
kmeans_mapreduce.py	Boucle itérative K-Means, critère de convergence, mode chunk et mode multi-serveurs
kmeans_classic.py	K-Means séquentiel de référence, timeout configurable
kmeans_streaming.py	Mode warm-start : flux IoT en vagues successives, centroïdes réutilisés entre vagues
benchmark.py	Benchmark comparatif sur 3 datasets ; timeout 20 s
simulation_comparison.py	Montée en charge 10k–1M (dashboard) ou 1k–2M (complet) ; simulation réseau IoT 1 Mbps ; timeout 15 s en mode complet

Chaque module a des dépendances minimales et peut être testé indépendamment. L'architecture favorise la maintenabilité et l'extension future.

4.2 Code du Mapper-Combiner et du Reducer

Le code implémente directement les phases MAP, COMBINE et REDUCE décrites en section 3. Le Mapper-Combiner s'exécute en parallèle via `multiprocessing.Pool` sur les W serveurs ; le Reducer s'exécute ensuite une seule fois pour produire les nouveaux centroïdes.

Détails d'implémentation :

- Le mapper parcourt chaque capteur du chunk, calcule sa distance à chaque centroïde via Euclidienne, et l'assigne au cluster le plus proche.
- Le combiner agrège les capteurs du même cluster en une seule entrée : somme des vecteurs de features et compteur d'éléments.
- Le reducer reçoit toutes les sommes partielles d'un cluster et les combine pour calculer le centroïde final.
- La convergence est vérifiée par la norme L2 entre centroïdes successifs : $\|c_k^{\text{old}} - c_k^{\text{new}}\|_2 < 10^{-4}$.

```

1 def mapper_combiner(args):    # called via Pool.map (single tuple argument)
2     chunk_lines, centroids = args[0], args[1]
3     partial = {}
4     for line in chunk_lines:
5         line = line.strip()
6         if not line or line.startswith("sensor_id"):
7             continue
8         parts = line.split(",")
9         try:
10            point = [float(parts[i]) for i in FEATURE_INDICES]
11        except (ValueError, IndexError):
12            continue
13        distances = [euclidean(point, c) for c in centroids]
14        cluster_id = distances.index(min(distances))
15        if cluster_id not in partial:
16            partial[cluster_id] = ([0.0]*len(point), 0)
17        s, cnt = partial[cluster_id]
18        partial[cluster_id] = ([s[d]+point[d] for d in range(len(point))], cnt+1)
19    return partial
20
21 def reducer(cluster_id, values):
22     total_sum, total_count = None, 0
23     for partial_sum, count in values:
24         total_sum = (list(partial_sum) if total_sum is None
25                      else [a+b for a,b in zip(total_sum, partial_sum)])
26         total_count += count
27     new_centroid = ([s/total_count for s in total_sum] if total_count > 0
28                    else total_sum or [0.0]*4)
29    return cluster_id, new_centroid, total_count

```

Listing 1: Mapper-Combiner et Reducer (kmeans_mr_job.py)

Orchestrateur : soumet chunks via `Pool.map`, collecte partiels, Shuffle par cluster, appelle Reducer. Convergence : $\|c_k^{\text{old}} - c_k^{\text{new}}\|_2 < 10^{-4}$.

4.3 Métriques de qualité et critères d'arrêt

Deux métriques sont utilisées pour évaluer la qualité du clustering :

Sum of Squared Errors (SSE) :

$$\text{SSE} = \sum_{k=1}^K \sum_{p \in C_k} \|p - c_k\|_2^2$$

Cette métrique mesure la compacité des clusters. Une SSE plus faible indique un clustering de meilleure qualité. Dans nos expériences, l'algorithme classique et MapReduce produisent la **même SSE**, confirmant l'exactitude mathématique.

Critère d'arrêt : L'algorithme converge quand la variation maximale des centroïdes entre deux itérations devient négligeable :

$$\max_k \|c_k^{(t+1)} - c_k^{(t)}\|_2 < \epsilon$$

Avec $\epsilon = 10^{-4}$ dans nos expériences. Ce seuil garantit une stabilité suffisante des résultats tout en limitant les itérations.

4.4 Exemple concret : 6 capteurs sur 2 serveurs

Pour valider le pipeline : 6 capteurs, $K = 2$, centroïdes initiaux (K-Means++) $C_0 = [35, 80, 0.5, 120]$ (rurale) et $C_1 = [22, 45, 2.5, 500]$ (urbaine). **MAP+COMBINE** : serveur A émet $\{S1, S2\} \rightarrow C_0$ et $\{S3\} \rightarrow C_1$; serveur B émet $\{S4\} \rightarrow C_0$ et $\{S5, S6\} \rightarrow C_1$ (2 sommes partielles/serveur). **Shuffle+Reduce** : agrégation globale $\rightarrow$ convergence en 2 itérations. Résultat : $\{S1, S2, S4\}$ (chaud, humide, rural) et $\{S3, S5, S6\}$ (froid, sec, urbain). **Résultat identique à l'algorithme classique.**

5 Résultats et performances

Évaluation comparative (temps d'exécution, scalabilité, qualité des clusters).

5.1 Benchmark sur données réelles

Évaluation comparative sur machine 4-CPU avec $K = 3$ clusters et 10 itérations max.

Dataset	Points	Classique (s)	MapReduce (s)	Ratio
Normal	1 500	0.123	0.124	1.01× — overhead quasi-nul
Stress test	50 000	2.375	1.004	2.37× — plus rapide
Multi-serveurs	1 500	0.074	0.166	2.24× — plus lent

Observations : Sur petit dataset (1 500 pts), l’overhead MapReduce est quasi-nul (0.124s vs 0.123s) : le pool de processus est créé très rapidement sur cette machine 4-CPU. Sur 50 000 capteurs, le parallélisme prend tout son avantage et MapReduce atteint un speedup de $2.37\times$ grâce au traitement parallèle sur 4 CPU.

$$\text{Speedup} = \frac{2.375}{1.004} \approx 2.37$$

5.2 Scalabilité — montée en charge

Test d’évolutivité jusqu’à 1 000 000 capteurs en mode dashboard (mesures réelles, données synthétiques, sans timeout artificiel). En mode complet, le timeout est fixé à 15 s (TIMEOUT_S=15 dans `simulation_comparison.py`).

Capteurs	Classique	MapReduce	Observation
10 000	0.426 s	0.374 s	MR déjà plus rapide (1.14×)
50 000	2.058 s	1.700 s	Parallélisme efficace (1.21×)
200 000	8.211 s	5.786 s	Speedup croissant (1.42×)
500 000	21.079 s (>15 s timeout)	14.008 s	Classique hors délai, MR termine
1 000 000	43.950 s (>15 s timeout)	29.734 s	Seul MapReduce peut traiter

MapReduce est plus rapide dès **10 000 capteurs**. Le gain croît régulièrement avec la taille : $1.14\times$ à 10k, $1.42\times$ à 200k, $1.51\times$ à 500k. À partir de 500 000 capteurs, le temps classique (21.079s) dépasse le timeout de 15s configuré dans `simulation_comparison.py` et l’algorithme ne convergerait pas. MapReduce, lui, reste viable jusqu’à 1 000 000 capteurs et au-delà. C’est un point critique : MapReduce devient l’**unique solution acceptable** pour les gros volumes.

5.3 Discussion des résultats

Overhead vs bénéfice. Sur cette machine 4-CPU, l’overhead de création du pool de processus est très faible (≈ 0.001 s à 1 500 pts : 0.124s vs 0.123s, mesuré par `benchmark.py`). MapReduce est déjà légèrement plus rapide à 10 000 capteurs (0.374s vs 0.426s), et le gain s’amplifie avec le volume.

Scalabilité linéaire. Entre 10 000 et 1 000 000 capteurs ($\times 100$ volume), le temps classique croît linéairement (0.426s $\rightarrow$ 43.950s, $\times 103$). MapReduce croît plus lentement (0.374s $\rightarrow$ 29.734s, $\times 79$), ce qui confirme le parallélisme efficace sur 4 CPU.

Seuil critique à 500 000. Avec le timeout de 15s de `simulation_comparison.py`, l’algorithme classique échouerait à partir de 500 000 capteurs (21.079s $>$ 15s). MapReduce, lui, continue de converger : c’est un point décisif où MapReduce devient non seulement plus rapide mais **indispensable**.

5.4 Qualité des clusters produits

Vérification que la parallélisation ne dégrade pas la qualité du résultat sur le dataset de 1 500 capteurs ($K = 3$).

Cluster	Centroïde	Taille	Profil identifié
C_0	[27.6°C, 59.3%, 1.08 m/s ² , 260 kbps]	524	Suburbain : conditions inter-médiaires
C_1	[34.3°C, 78.3%, 0.53 m/s ² , 124 kbps]	525	Zone rurale/forestière : chaude, humide
C_2	[22.2°C, 45.2%, 2.49 m/s ² , 512 kbps]	451	Urbain industriel : vibration, trafic élevés

La somme des erreurs quadratiques (SSE) est identique : $SSE_{\text{classique}} = SSE_{\text{MapReduce}} = 3\,550\,130.6$. Les clusters correspondent à des profils IoT cohérents. **Conclusion : pas de perte de qualité.**

5.5 Limites et perspectives

L'approche actuelle repose sur plusieurs hypothèses et pourrait être étendue de plusieurs façons.

Limites : (1) L'implémentation utilise `multiprocessing.Pool` sur une seule machine ; une vraie distribution sur réseau (Spark, Hadoop) améliorerait la scalabilité. (2) Les données sont supposées tenir en mémoire après chunking ; pour des streaming massifs, adapter l'algorithme à un paradigme en flux serait nécessaire. (3) K est fixe ; des algorithmes de sélection automatique de K (silhouette, coude) pourraient être intégrés.

Perspectives : (1) Porter l'implémentation sur Apache Spark pour bénéficier de distribution complète et tolérance aux pannes. (2) Implémenter K-Means++ distribué (actuellement centralisé). (3) Ajouter d'autres métriques de qualité (Davies-Bouldin, Calinski-Harabasz) et d'autres initializations (k-means||). (4) Évaluer la convergence avec des critères d'arrêt adaptatifs.

6 Conclusion

Ce travail a mis en œuvre et évalué le paradigme MapReduce pour le clustering K-Means sur un réseau de capteurs IoT distribués. Les résultats obtenus permettent de dégager quatre enseignements principaux.

1. Exactitude mathématique. MapReduce produit exactement le même résultat que l'algorithme classique. La SSE est identique (3 550 130.6), car l'agrégation par sommes partielles est une transformation mathématiquement exacte. La parallélisation ne sacrifie donc aucune précision.

2. Seuil de rentabilité. MapReduce est avantageux dès 10 000 capteurs sur cette machine 4-CPU (l'overhead de démarrage du pool est de l'ordre du milliseconde). Le speedup

atteint $2.37\times$ à 50 000 capteurs (benchmark sur `large_sensors.csv`) et se stabilise autour de $1.5\times$ lors de la montée en charge jusqu'à 1 000 000 capteurs.

3. Efficacité réseau du Combiner. Le Combiner est un facteur clé. En réduisant le volume de N enregistrements à K sommes partielles, il divise le trafic d'un facteur N/K : de $\times 100$ à l'échelle testée jusqu'à $\times 3\,333$ à 50 000 capteurs par serveur.

4. Scalabilité au-delà de 500 000 capteurs. MapReduce est la seule option viable pour les gros volumes. Avec le timeout de 15 s configuré dans `simulation_comparison.py`, l'algorithme classique échouerait à partir de 500 000 capteurs ($21.079\text{s} > 15\text{s}$), tandis que MapReduce termine dans tous les cas testés, jusqu'à 1 000 000 capteurs (29.734s).

En conclusion, MapReduce pour K-Means constitue une solution viable et scalable pour le clustering IoT distribué, avec garantie d'exactitude et gains significatifs dès que le volume de données devient substantiel.